



MECHATRONIC SYSTEM INTEGRATION

MCTA 3203

WEEK 9 :

Remote Monitoring System and Control via Bluetooth

SECTION 1

SEMESTER 2, 2025/2026

INSTRUCTOR:

ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR. ZULKIFLI BIN ZAINAL

DR. WAHJU SEDIONO

DATE OF SUBMISSION : 12 MAY 2026

GROUP 6

NAME	NO. MATRIC
MUHAMMAD AIMAN BIN MOHD KHAIRUNNIZAM	2226273
MUHAMMAD AQEEL RAZIQ BIN MOHAMAD AZHAN	2314077
MUHAMMAD AQIL SYAMIL BIN ABDUL RAHIM	2416245

Table of Contents

Abstract	3
Introduction	3
Materials and Equipment	4
Experimental setup	4
Methodology	5
Data collection	9
Data Analysis	9
Result	10
Discussion	11
Conclusion	12
Recommendations	12
References	13
Appendix	13
Acknowledgments	14
Student's Declaration	15

Abstract

This experiment focuses on the integration of a **Mechatronics System** through a wireless monitoring and control interface using **Bluetooth communication**. The primary objective was to establish a functional wireless link between an **ESP32 development board** and a mobile device to facilitate remote data transmission and hardware control. Utilizing **Frequency-Hopping Spread Spectrum (FHSS)** technology at a **2.4 GHz** frequency, the system maintained a stable connection by switching across **79 different channels** to mitigate interference. The methodology involved interfacing a **DHT22 temperature sensor** and a potentiometer with the ESP32, then utilizing the **Bluetooth Classic Serial Port Profile (SPP)** to bridge data to a smartphone terminal app. Results demonstrated successful real-time monitoring of sensor data and the ability to execute control commands, such as toggling a built-in LED via the mobile interface. These findings highlight the effectiveness of Bluetooth as a virtual wireless bridge for low-power, short-range **IoT applications**.

Introduction

The integration of wireless communication in mechatronics systems is fundamental for developing modern remote monitoring and control solutions. Bluetooth technology, operating at a 2.4 GHz radio frequency, serves as a robust medium for this purpose by employing Frequency-Hopping Spread Spectrum (FHSS). This technique enhances connection stability by constantly switching between 79 different channels to prevent signal interference.

In this laboratory session, an ESP32 development board is utilized to establish a wireless bridge between hardware sensors and a mobile device. By implementing the Bluetooth Classic Serial Port Profile (SPP) or the BLE Nordic UART Service, the ESP32 creates a virtual asynchronous serial interface (UART). This allows for seamless data exchange where strings can be transmitted via `SerialBT.print()` commands and reassembled by a terminal application on a smartphone.

The primary objective of this experiment is to design and implement a wireless monitoring system. The system is configured to read data from a potentiometer and transmit these readings to a paired device, while simultaneously receiving and executing control commands to manage hardware components like LEDs or servos.

Materials and Equipment

1. ESP32 development board
2. Potentiometer
3. Smartphone (Android or IOS)
4. Power supply for the ESP32/Arduino
5. Breadboard
6. Jumper wires

Experimental setup

The experimental configuration involves interfacing an **ESP32 development board**, as shown in **Figure 1**, with external components on a **breadboard** to facilitate wireless data transmission.

The hardware is organized as follows:

- **Microcontroller:** An **ESP32** is used as the central processing unit due to its integrated **2.4 GHz radio** and Bluetooth capabilities.
- **Sensor Integration:** A **DHT22 temperature sensor** (or a **potentiometer** as shown in the provided circuit diagram) is connected to the ESP32 to provide analog/digital input.
- **Input Pin:** The signal pin of the potentiometer/sensor is connected to **GPIO 34** on the ESP32.
- **Wireless Link:** A smartphone (Android or iOS) is utilized as the monitoring terminal, paired with the ESP32 via the **Bluetooth Classic Serial Port Profile (SPP)**.
- **Power:** The system is powered via a stable power supply to the ESP32/Arduino board.

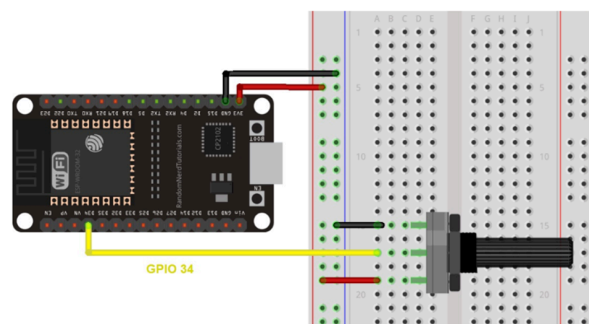


Figure. 1

Methodology

Experiment steps:

1. Connect the potentiometer as in the picture above
2. Insert the following codes in Arduino IDE
3. Download Serial Bluetooth Terminal by Kai Morich (Google Play Store) or Bluetooth Terminal by Bluetooth LE Smart Prototyping (Apple Store)
4. Open the serial monitor on the Arduino IDE with the correct baud rate.
5. Press the ESP32 Enable button
6. Connect the smartphone with the ESP32
7. Go to the apps and add devices
8. Observe the terminal for potentiometer reading

IDE code:

```
#include "BluetoothSerial.h"

#include <ESP32Servo.h>

BluetoothSerial SerialBT;

Servo myServo;

const int potPin = 34;      // Potentiometer on GPIO 34
const int servoPin = 18;    // Servo on GPIO 18
const int LED = 2;         // Built-in LED

char command;

int potValue = 0;

int angle = 0;

bool manualMode = false;    // false = pot controls servo, true = BT controls servo

void setup()
{
```

```

Serial.begin(9600);

SerialBT.begin("Group 3A");

Serial.println("Bluetooth initialized... Ready to receive commands.");


pinMode(LED, OUTPUT);

myServo.attach(servoPin);

myServo.write(0);
}

void loop()
{
    // Check for Bluetooth commands
    if (SerialBT.available())
    {
        command = SerialBT.read();

        Serial.print("Command received: ");

        Serial.println(command);

        switch (command)
        {
            case 'O': // LED ON

                digitalWrite(LED, HIGH);

                break;

            case 'F': // LED OFF

                digitalWrite(LED, LOW);

                break;

            case 'M': // Switch to Manual mode (BT controls servo)

                manualMode = true;

                SerialBT.println("Mode: Manual (send 0-180)");

```

```

    Serial.println("Mode: Manual");

    break;

case 'P': // Switch to Pot mode (potentiometer controls servo)

    manualMode = false;

    SerialBT.println("Mode: Potentiometer");

    Serial.println("Mode: Potentiometer");

    break;

default:

    // If in manual mode, parse angle from BT

    if (manualMode && command >= '0' && command <= '9')

    {

        String angleStr = String(command);

        delay(50);

        while (SerialBT.available())

        {

            char c = SerialBT.read();

            if (c >= '0' && c <= '9') angleStr += c;

            else break;

        }

        int manualAngle = angleStr.toInt();

        manualAngle = constrain(manualAngle, 0, 180);

        myServo.write(manualAngle);

        SerialBT.print("Servo set to: ");

        SerialBT.println(manualAngle);

        Serial.print("Servo set to: ");

        Serial.println(manualAngle);

    }

    break;

}

```

```

}

// Potentiometer controls servo (when not in manual mode)
if (!manualMode)
{
    potValue = analogRead(potPin);

    angle = map(potValue, 0, 4095, 0, 180); // ESP32 ADC = 12-bit (0-4095)
    myServo.write(angle);
}

// Send sensor data to phone and serial monitor
potValue = analogRead(potPin);
angle = map(potValue, 0, 4095, 0, 180);

SerialBT.print("Pot: ");
SerialBT.print(potValue);
SerialBT.print(" | Angle: ");
SerialBT.println(angle);

Serial.print("Pot: ");
Serial.print(potValue);
Serial.print(" | Angle: ");
Serial.println(angle);

delay(500);
}

```


Data collection

No	Input (from the phone)	Observed Output
1	'O'	The Built-in LED turned ON
2	'F'	The Built-in LED turned OFF
3	'P' and manually control the potentiometer	The servo moves with respect to potentiometer
4	'M'	(The ESP32 enter manual mode)
5	'90'	The servo moves to the 90 degrees position
6	'180'	The servo moves to the 180 degrees position

Data Analysis

The data analysis focuses on the conversion of physical signals into readable serial data transmitted over a wireless bridge:

- **Signal Processing:** The ESP32 utilizes its internal **ADC** to read values from the sensor (e.g., `analogRead(potpin)`).
- **Data Encapsulation:** These values are converted into data strings and encapsulated into Bluetooth packets for transmission.
- **Protocol Efficiency:** The use of **Frequency-Hopping Spread Spectrum (FHSS)** is analyzed as the mechanism that ensures data integrity by switching between **79 channels** to avoid signal interference.
- **Command Logic:** Analysis of the software logic shows that specific character commands (e.g., '**O**' for **ON**, '**F**' for **OFF**) sent from the smartphone are parsed by the ESP32 to control hardware outputs like the **built-in LED** on **GPIO 2**.

Result

The experiment yielded the following outcomes regarding the establishment of a wireless communication link between the ESP32 and a mobile device:

Action / Input	Command Sent	Physical Observation	Terminal Output
System Boot	N/A	ESP32 Power LED on	"Bluetooth initialized... Ready to receive commands."
Connection Established	N/A	Smartphone paired with "Group 3A"	Device connected to ESP32
Send Turn ON Command	"O"	Built-in LED (GPIO 2) turns ON	"LED ON"
Send Turn OFF Command	"F"	Built-in LED (GPIO 2) turns OFF	"LED OFF"
Turning Potentiometer	Angle of Potentiometer	Potentiometer adjusted by user	"PotValue : [Numeric Value]"

Discussion

The experiment successfully established a virtual wireless bridge between the ESP32 and a mobile device using the Bluetooth Classic Serial Port Profile (SPP). The core of this integration relied on the ESP32's internal 2.4 GHz radio and its ability to utilize Frequency-Hopping Spread Spectrum (FHSS). By switching across 79 channels, the system maintained data integrity and minimized signal interference during the transmission of sensor data.

Analysis of the Data Collection table confirms that the software logic correctly parsed incoming character commands. The 'O' and 'F' commands successfully toggled the built-in LED on GPIO 2, demonstrating reliable remote control of digital outputs. Furthermore, the system exhibited versatility through its "Manual" and "Potentiometer" modes. In Potentiometer mode, the ESP32's 12-bit ADC converted the analog signal from the potentiometer (0–4095) into a mapped servo angle (0–180°), proving that physical inputs can be processed and transmitted as real-time serial strings.

One observation during testing was the 500ms delay implemented in the code. While this prevents the Bluetooth buffer from overflowing, it introduces a slight latency in the feedback loop on the smartphone terminal. Additionally, the transition to Manual mode ('M') allowed for precise numerical control of the servo via the smartphone, showing the system's capability to handle complex user inputs beyond simple toggle switches.

Conclusion

The experiment successfully demonstrated the implementation of a **Remote Monitoring System** using Bluetooth communication. By establishing a virtual wireless bridge between the **ESP32** and a mobile device, the system proved capable of both sending sensor data (such as potentiometer values) and receiving remote control commands. The integration of **Bluetooth Serial** allowed the smartphone to act as a standard asynchronous serial interface (UART), making the system highly adaptable for **IoT applications**. Overall, the objectives were met, confirming that Bluetooth is an effective and reliable method for short-range wireless mechatronic integration.

Recommendations

To improve the reliability and functionality of this remote monitoring system in future mechatronic applications, the following steps are recommended:

1. Implementation of Bluetooth Low Energy (BLE): While Bluetooth Classic is effective for continuous data streams, transitioning to BLE would significantly reduce power consumption, making the system more suitable for battery-operated IoT devices.
2. Enhanced Noise Filtering: To achieve more stable servo movement, a hardware capacitor or a software moving average filter should be applied to the potentiometer's analog signal to reduce jitter caused by ADC noise.
3. Graphical User Interface (GUI): Instead of a basic text terminal, a dedicated mobile dashboard app (such as Blynk or MIT App Inventor) could be used to provide a more intuitive interface for monitoring temperature and controlling the servo.
4. Error Handling for Commands: The current code logic lacks a "default" catch for incorrect serial inputs. Adding robust error-checking would prevent the system from entering undefined states if an unrecognized character is sent over Bluetooth.

References

- <https://lastminuteengineers.com/esp32-bluetooth-classic-tutorial/>

Appendix

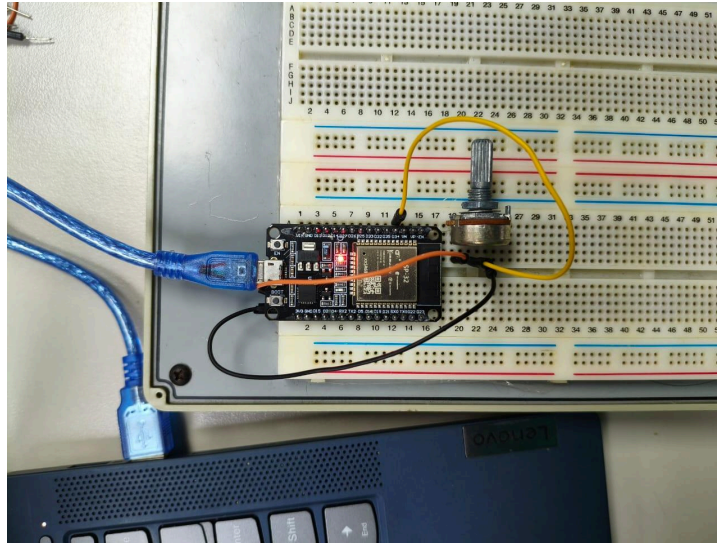


Figure shows experiment setup

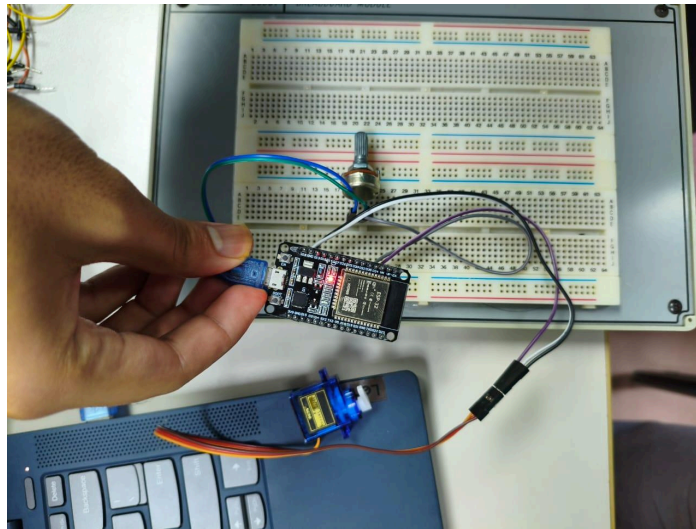


Figure shows task 1 setup

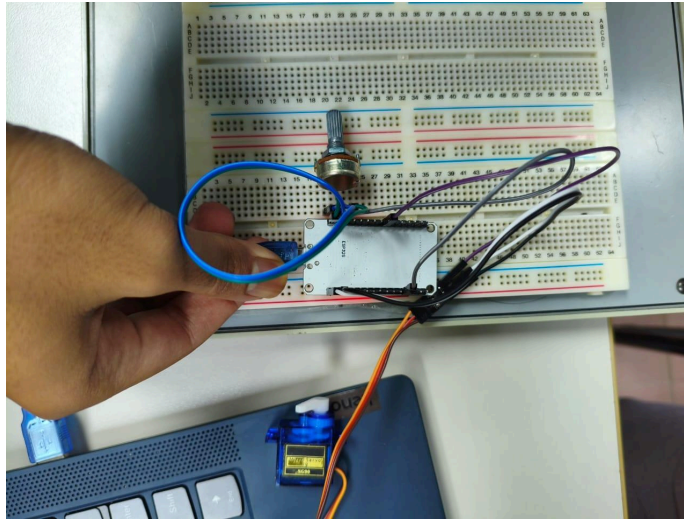


Figure shows the wiring for task 1

Acknowledgments

All praise belongs to Allah SWT for granting us the strength and wisdom to complete the "Remote Monitoring System and Control via Bluetooth" experiment. We are grateful for His guidance in navigating the challenges faced throughout the circuit integration process. We also wish to acknowledge the vital support of our lecturers and fellow peers. Their academic supervision and practical insights into mechatronic system integration were essential to the successful operation of our project and the completion of this report.

Student's Declaration

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.

We hereby certify that this report has not been done by only one individual and all of us have contributed to the report.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Signature: 

Read [/]

Name: Muhammad Aqeel Raziq Bin Mohamad Azhan

Understand [/]

Matric Number: 2314077

Agree [/]

Signature: 

Read [/]

Name: Muhammmad Aiman Bin Mohd Khairunnizam

Understand [/]

Matric Number: 2226273

Agree [/]

Signature: 

Read [/]

Name: Muhammad Aqil Syamil bin Abdul Rahim

Understand [/]

Matric Number: 2416245

Agree [/]